

Object Oriented Programming (Multiple Inheritance)

Abbas Khalid

April 21, 2025

Table of contents

1 Multiple Inheritance

2 Virtual Base Classes

Multiple Inheritance

- A class can be derived from more than one base class. This is called multiple inheritance. For example, the syntax for a class that is derived from two base classes is:

```
1 class Child : mode Parent1, mode Parent2 {  
2 // body of Child Class  
3 };
```

- When two base classes have functions with the same name, while a class derived from both base classes has no function with this name, the compiler can't figure out which of the two functions is meant.
- Another ambiguity arises if we derive a class from two classes that are each derived from the same class. This creates a diamond-shaped inheritance tree and is known as a diamond problem.

Example 1

```
1 class Father {
2     public:
3         void message(){cout<<" Father Class."<<endl;}
4 };
5
6 class Mother {
7     public:
8         void message(){cout<<" Mother Class."<<endl;}
9 };
10
11 class Child : public Father, public Mother{
12     public:
13         void message(){cout<<" Child Class."<<endl;}
14 };
15
16 int main(){
17
18     Child C;
19     C.message();
20
21     return 0;
22 }
```

Child Class.

Example 2

```
1  class Father {
2      public:
3          void message(){cout<<" Father Class."<<endl;}
4  };
5
6  class Mother {
7      public:
8          void message(){cout<<" Mother Class."<<endl;}
9  };
10
11 class Child : public Father, public Mother{
12     public:
13         void message() {
14             cout<<" Child Class."<<endl;
15             Father::message(); Mother::message();
16         }
17 };
18
19 int main(){
20     Child C;    C.message();
21     return 0;
22 }
```

Child Class.
Father Class.
Mother Class.

Example 3

```
1 class Father {
2     public:
3         void message(){cout<<" Father Class."<<endl;}
4 };
5
6 class Mother {
7     public:
8         void message(){cout<<" Mother Class."<<endl;}
9 };
10
11 class Child : public Father, public Mother{
12
13 };
14
15 int main(){
16
17     Child C;
18     C.message(); // Ambiguity
19
20 return 0;
21 }
```

Example 4

```
1 class Father {
2     public:
3         void message(){cout<<" Father Class."<<endl;}
4 };
5
6 class Mother : public Father {
7
8 };
9
10 class Child : public Father {
11
12 };
13
14 class GrandChild : public Mother, public Child {
15
16 };
17
18 int main(){
19     GrandChild GC;
20     GC.message(); // Error! Ambiguity (Diamond Problem)
21 return 0;
22 }
```

Example 5

```
1 class Father {
2     public:
3         void message(){cout<<" Father Class."<<endl;}
4 };
5
6 class Mother : public Father {
7
8 };
9
10 class Child : public Father {
11
12 };
13
14 class GrandChild : public Mother, public Child {
15
16 };
17
18 int main(){
19     GrandChild GC;
20     GC.Child::message();      GC.Mother::message();
21     return 0;
22 }
```

Father Class.
Father Class.

Constructors and Destructors in Multiple Inheritance

- We know the syntax for a class that is derived from two base classes is:

```
1 class Child : mode Parent1, mode Parent2 {  
2 // body of Child Class  
3 };
```

- When object of a Child class is created, the constructor of the Child class is called. The names of the Parent class constructors follow the colon and are separated by commas. When the Child class constructor is invoked, these Parent class constructors will be executed.

```
1 Child (parameterList):Parent1 (argumentList),  
    Parent2 (argumentList) {  
2 // body of Child Class Constructor  
3 }
```

- Note that Parent class constructors are called in the order in which Parent classes are written during inheritance (left to right) e.g., here constructor of class Parent1 will be invoked first. The argumentList must match the parameterList of the respective Parent class constructor.
- The destructors execute in opposite order.

Example 1

```
1 class Father {
2     public:
3         Father(){cout<<"Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother {
8     public:
9         Mother(){cout<<"Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : public Mother, public Father{
14     public:
15         Child(){cout<<"Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
18
19 int main(){
20     Child C;
21     return 0;
22 }
```

Mother Class Constructor.
Father Class Constructor.
Child Class Constructor.
Child Class Destructor.
Father Class Destructor.
Mother Class Destructor.

Example 2

```
1  class Father {
2      public:
3          Father(){cout<<"Father Class Constructor."<<endl;}
4          ~Father(){cout<<" Father Class Destructor."<<endl;}
5  };
6
7  class Mother {
8      public:
9          Mother(){cout<<"Mother Class Constructor."<<endl;}
10         ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11  };
12
13  class Child : public Father, public Mother{
14      public:
15          Child(){cout<<"Child Class Constructor."<<endl;}
16          ~Child(){cout<<" Child Class Destructor."<<endl;}
17  };
18
19  int main(){
20      Child C;
21      return 0;
22  }
```

Father Class Constructor.
Mother Class Constructor.
Child Class Constructor.
Child Class Destructor.
Mother Class Destructor.
Father Class Destructor.

Example 3

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 3 (Cont.)

```
1 class GrandChild : public Mother, public Child{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

```
Father Class Constructor.
Mother Class Constructor.
Father Class Constructor.
Child Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Child Class Destructor.
Father Class Destructor.
Mother Class Destructor.
Father Class Destructor.
```

Example 4

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 4 (Cont.)

```
1 class GrandChild : public Child, public Mother{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

```
Father Class Constructor.
Child Class Constructor.
Father Class Constructor.
Mother Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Mother Class Destructor.
Father Class Destructor.
Child Class Destructor.
Father Class Destructor.
```

Scenario 9.1

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 9.1 (Cont.)

```
1 class Worker {
2     string jobTitle;
3     double salary;
4
5     public:
6         Worker(string job, double sal):jobTitle(job),salary(sal)
7             {}
8
9         void setJobTitle(string job) { jobTitle = job; }
10        void setSalary(double sal) { salary = sal; }
11
12        string getJobTitle() const { return jobTitle; }
13        double getSalary() const { return salary; }
14
15        void printInfo() const {
16            cout << "Job Title: " << jobTitle << "\n";
17            cout << "Salary: $" << salary << "\n";
18        }
19    };
```

Scenario 9.1 (Cont.)

```
1 class Student : public Person, public Worker {
2     int studentId;
3     string grade;
4
5     public:
6         Student(string name,int age,string job,double sal,int Id,
7             string grade):Person(name,age),Worker(job,sal),
8             studentId(Id),grade(grade) {}
9
10    void setStudentId(int Id) { studentId = Id; }
11    void setGrade(string grade) { this->grade = grade; }
12
13    int getStudentId() const { return studentId; }
14    string getGrade() const { return grade; }
15
16    void printInfo() const {
17        Person::printInfo(); // Call Person's method
18        Worker::printInfo(); // Call Worker's method
19        cout << "Student ID: " << studentId << "\n";
20        cout << "Grade: " << grade << "\n";
21    }
22};
```

Scenario 9.1 (Cont.)

```
1 int main() {  
2  
3     Student S("Imran Ali", 25, "Programmer", 750, 8288151, "A");  
4     S.printInfo();  
5  
6     return 0;  
7 }
```

```
Name: Imran Ali  
Age: 25  
Job Title: Software Engineer  
Salary: $750  
Student ID: 8288151  
Grade: A
```

Scenario 9.2: Diamond Problem

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 9.2 (Cont.)

```
1 class Student : public Person {
2     int studentId;
3     string grade;
4
5 public:
6     Student(string name, int age, int Id, string grade):
7         Person(name, age), studentId(Id), grade(grade) {}
8
9     void setStudentId(int Id) { studentId = Id; }
10    void setGrade(string grade) { this->grade = grade; }
11
12    int getStudentId() const { return studentId; }
13    string getGrade() const { return grade; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Student ID: " << studentId << "\n";
18        cout << "Grade: " << grade << "\n";
19    }
20};
```

Scenario 9.2 (Cont.)

```
1 class Worker : public Person {
2     string jobTitle;
3     double salary;
4
5 public:
6     Worker(string name, int age, string job, double sal):
7         Person(name, age), jobTitle(job), salary(sal) {}
8
9     void setJobTitle(string job) { jobTitle = job; }
10    void setSalary(double salary) { this->salary = salary; }
11
12    string getJobTitle() const { return jobTitle; }
13    double getSalary() const { return salary; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Job Title: " << jobTitle << "\n";
18        cout << "Salary: $" << salary << "\n";
19    }
20 };
```

Scenario 9.2 (Cont.)

```
1 class WorkingStudent : public Student, public Worker {
2     int studentId;
3     string grade;
4
5     public:
6         WorkingStudent(string name,int age,string job,double sal,
7             int Id,string grade):Student(name,age,Id,grade),
8             Worker(name,age,job,sal),studentId(Id),grade(grade)
9         {}
10 };
11
12 int main() {
13
14     WorkingStudent WS("Imran Ali",25,"Programmer",750,8288151,"A");
15     WS.printInfo(); // Ambiguous Call
16
17     return 0;
18 }
```

Scenario 9.3: Solution to the Diamond Problem

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 9.3 (Cont.)

```
1 class Student : public Person {
2     int studentId;
3     string grade;
4
5 public:
6     Student(string name, int age, int Id, string grade):
7         Person(name, age), studentId(Id), grade(grade) {}
8
9     void setStudentId(int Id) { studentId = Id; }
10    void setGrade(string grade) { this->grade = grade; }
11
12    int getStudentId() const { return studentId; }
13    string getGrade() const { return grade; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Student ID: " << studentId << "\n";
18        cout << "Grade: " << grade << "\n";
19    }
20};
```

Scenario 9.3 (Cont.)

```
1 class Worker : public Person {
2     string jobTitle;
3     double salary;
4
5 public:
6     Worker(string name, int age, string job, double sal):
7         Person(name, age), jobTitle(job), salary(sal) {}
8
9     void setJobTitle(string job) { jobTitle = job; }
10    void setSalary(double salary) { this->salary = salary; }
11
12    string getJobTitle() const { return jobTitle; }
13    double getSalary() const { return salary; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Job Title: " << jobTitle << "\n";
18        cout << "Salary: $" << salary << "\n";
19    }
20 };
```

Scenario 9.3 (Cont.)

```
1 class WorkingStudent : public Student, public Worker {
2     int studentId;
3     string grade;
4
5     public:
6     WorkingStudent(string name,int age,string job,double sal,
7         int Id,string grade):Student(name,age,Id,grade),
8         Worker(name,age,job,sal),studentId(Id),grade(grade)
9         {}
10
11     void printInfo() const {
12         //Resolving ambiguity by specifying which base class's method to call
13         Student::printInfo();
14         Worker::printInfo();
15     }
16 };
```

Scenario 9.3 (Cont.)

```
1 int main() {  
2  
3     WorkingStudent WS("Imran Ali",25,"Programmer",750,8288151,"A");  
4     WS.printInfo(); // OK now.  
5  
6     return 0;  
7 }
```

```
Name: Imran Ali  
Age: 25  
Student ID: 8288151  
Grade: A  
Name: Imran Ali  
Age: 25  
Job Title: Software Engineer  
Salary: $750
```

Virtual Base Classes

- We have already seen that ambiguity introduced in multiple inheritance can be resolved with the help of scope resolution operator. However, this technique does not prevent two copies to be inherited.
- When two or more objects are derived from a common base class, we can prevent multiple copies of the base class from being present in the derived class by declaring the base class as **virtual** when it is inherited.
- We accomplish this by preceding the base class' name with the keyword **virtual** when it is inherited.

```
1 class Child : virtual mode Parent {  
2 // body of Child Class  
3 };
```

```
1 class Child : mode virtual Parent {  
2 // body of Child Class  
3 };
```

- With this declaration, the *Parent* class becomes the virtual base class, and *Child* class inherits virtually from *Parent* class.
- The **virtual** keyword only needs to be used in the direct inheritance chain from the virtual base class. Subsequent classes in the hierarchy inherit virtually implicitly.

Example 1

```
1 class Father {
2     public:
3         void message(){cout<<"Father Class."<<endl;}
4 };
5
6 class Mother : virtual public Father {
7
8 };
9
10 class Child : virtual public Father {
11
12 };
13
14 class GrandChild : public Mother, public Child {
15
16 };
17 int main(){
18     GrandChild GC;      GC.message();
19     GC.Child::message(); // message() from the Child class
20     GC.Mother::message(); // message() from Mother class
21 return 0;
22 }
```

Father Class.
Father Class.
Father Class.

Example 2

```
1 class Father {
2     public:
3         void message(){cout<<"Father Class."<<endl;}
4 };
5
6 class Mother : virtual public Father {
7
8 };
9
10 class Child : virtual public Father {
11
12 };
13
14 class GrandChild : public Child, public Mother{
15
16 };
17 int main(){
18     GrandChild GC;      GC.message();
19     GC.Child::message(); // message() from the Child class
20     GC.Mother::message(); // message() from Mother class
21 return 0;
22 }
```

Father Class.
Father Class.
Father Class.

Example 3

```
1 class Father {
2     protected: int i;
3 };
4 class Mother : virtual public Father {
5     public:
6         void set(int a){i = a;}
7         void display() {cout<<" Mother Class:"<<i<<endl;}
8 };
9 class Child : virtual public Father {
10     public:
11         void set(int a){i = a;}
12         void display() {cout<<" Child Class:"<<i<<endl;}
13 };
14 class GrandChild : public Mother, public Child {
15     public:
16         void display(){Child::display();Mother::display();}
17 };
18 int main(){
19     GrandChild GC;  GC.Child::set(200); // (Single copy)
20     GC.Mother::set(100);    GC.display();
21     return 0;
22 }
```

Child Class:100
Mother Class:100

Constructors and Destructors in Virtual Base Classes

- When we use the *virtual* keyword in the inheritance declaration, multiple paths in the inheritance hierarchy lead to the same virtual base class (through different intermediate classes), and only one instance of that virtual base class is created in the most derived class.
- When using virtual inheritance, all intermediate classes in the hierarchy that lead to a common virtual base class should inherit that base class virtually to avoid redundancy and ambiguity.
- If any of the intermediate classes does not inherit *Parent* virtually, it would create a separate instance of *Parent*, leading to ambiguity.

Constructors and Destructors in Virtual Base Classes

- When we have a mix of virtual and non-virtual base classes, the sequence of constructor calls during the creation of an object follows a specific order.
 - 1 First, the constructors of virtual base classes are called in the order of their appearance in the inheritance hierarchy from the most derived class. Each virtual base class is constructed only once, regardless of how many paths lead to it.
 - 2 After the virtual base class constructors, the constructors of non-virtual base classes are called in the order of their appearance in the inheritance hierarchy from the most derived class.
 - 3 Finally, the constructor of the most derived class executes.

Example 1

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : virtual public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : virtual public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 1 (Cont.)

```
1 class GrandChild : public Mother, public Child{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

Father Class Constructor.
Mother Class Constructor.
Child Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Child Class Destructor.
Mother Class Destructor.
Father Class Destructor.

Example 2

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : virtual public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : virtual public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 2 (Cont.)

```
1 class GrandChild : public Child, public Mother{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

Father Class Constructor.
Child Class Constructor.
Mother Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Mother Class Destructor.
Child Class Destructor.
Father Class Destructor.

Example 3

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : virtual public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 3 (Cont.)

```
1 class GrandChild : public Child, public Mother{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

Father Class Constructor.
Father Class Constructor.
Child Class Constructor.
Mother Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Mother Class Destructor.
Child Class Destructor.
Father Class Destructor.
Father Class Destructor.

Example 4

```
1 class Father {
2     public:
3         Father(){cout<<" Father Class Constructor."<<endl;}
4         ~Father(){cout<<" Father Class Destructor."<<endl;}
5 };
6
7 class Mother : public Father {
8     public:
9         Mother(){cout<<" Mother Class Constructor."<<endl;}
10        ~Mother(){cout<<" Mother Class Destructor."<<endl;}
11 };
12
13 class Child : virtual public Father{
14     public:
15         Child(){cout<<" Child Class Constructor."<<endl;}
16         ~Child(){cout<<" Child Class Destructor."<<endl;}
17 };
```

Example 4 (Cont.)

```
1 class GrandChild : public Child, public Mother{
2     public:
3         GrandChild(){cout<<"GrandChild Class Constructor.\n";}
4         ~GrandChild(){cout<<"GrandChild Class Destructor.\n";}
5 };
6
7 int main(){
8
9     GrandChild GC;
10
11     return 0;
12 }
```

Father Class Constructor.
Child Class Constructor.
Father Class Constructor.
Mother Class Constructor.
GrandChild Class Constructor.
GrandChild Class Destructor.
Mother Class Destructor.
Father Class Destructor.
Child Class Destructor.
Father Class Destructor.

Example 5

```
1 class Parent {
2     protected:
3         int data;
4     public:    //Virtual class constructor is always called first
5         Parent(int value) : data(value) //called once, single copy
6             {cout<<"Parent Class: "<<data<<endl;} // shared
7 };
8
9 // The following makes Parent class 'virtual'
10 class Inter1 : virtual public Parent {
11     public:
12         Inter1(int value):Parent(value)
13             {cout<<"Inter1 Class:"<<data<<endl;}
14 };
15
16 class Inter2 : virtual public Parent {
17     public:
18         Inter2(int value):Parent(value)
19             {cout<<"Inter2 Class:"<<data<<endl;}
20 };
```

Example 5 (Cont.)

```
1 //In the declaration below order matters
2 class Child : public Inter2, public Inter1 {
3     public: //Inside the constructor below order does not matter
4         Child(int a, int b):Parent(0),Inter1(a),Inter2(b)
5         {cout<<" Child Class: " <<data<<endl;}
6         int getData(){return data;}
7 };
8 /* As soon as the most derived class object is instantiated, the virtual base
   class constructor executes first. i.e. First, the Parent constructor
   then the Inter2, and finally the Inter1 constructor executes */
9
10 int main() {
11     Child obj(10,20);
12     cout<<"Data in Child: " <<obj.getData()<<endl;
13     return 0;
14 }
```

```
Parent Class: 0
Inter2 Class: 0
Inter1 Class: 0
Child Class: 0
Data in Child: 0
```

Example 6

```
1 class Parent {
2     protected:
3         int data;
4     public:    //Virtual class constructor is always called first
5         Parent(int value) : data(value) //called once, single copy
6             {cout<<"Parent Class: "<<data<<endl;} // shared
7 };
8
9 // The following makes Parent class 'virtual'
10 class Inter1 : virtual public Parent {
11     public:
12         Inter1(int value):Parent(value)
13         {data = value; cout<<"Inter1 Class:"<<data<<endl;}
14 };
15
16 class Inter2 : virtual public Parent {
17     public:
18         Inter2(int value):Parent(value)
19         {data = value; cout<<"Inter2 Class:"<<data<<endl;}
20 };
```

Example 6 (Cont.)

```
1 //In the declaration below order matters
2 class Child : public Inter2, public Inter1 {
3     public: //Inside the constructor below order does not matter
4         Child(int a, int b):Parent(0),Inter1(a),Inter2(b)
5         {cout<<" Child Class: " <<data<<endl;}
6         int getData(){return data;}
7 };
8 /* As soon as the most derived class object is instantiated, the virtual base
   class constructor executes first. i.e. First, the Parent constructor
   then the Inter2, and finally the Inter1 constructor executes */
9
10 int main() {
11     Child obj(10,20);
12     cout<<"Data in Child: " <<obj.getData()<<endl;
13     return 0;
14 }
```

```
Parent Class: 0
Inter2 Class: 20
Inter1 Class: 10
Child Class: 10
Data in Child: 10
```

Example 7

```
1 class Parent {
2     protected:
3         int data;
4     public:    //Virtual class constructor is always called first
5         Parent(int value) : data(value) //called once, single copy
6             {cout<<"Parent Class: "<<data<<endl;} // shared
7 };
8
9 // The following makes Parent class 'virtual'
10 class Inter1 : virtual public Parent {
11     public:
12         Inter1(int value):Parent(value)
13         { cout<<"Inter1 Class:"<<data<<endl;}
14 };
15
16 class Inter2 : virtual public Parent {
17     public:
18         Inter2(int value):Parent(value)
19         {cout<<"Inter2 Class:"<<data<<endl;}
20 };
```

Example 7 (Cont.)

```
1 //In the declaration below order matters
2 class Child : public Inter1, public Inter2 {
3     public: //Inside the constructor below order does not matter
4         Child(int a, int b):Parent(0),Inter2(b),Inter1(a)
5         {cout<<" Child Class: " <<data<<endl;}
6         int getData(){return data;}
7 };
8 /* As soon as the most derived class object is instantiated, the virtual base
   class constructor executes first. i.e. First, the Parent constructor
   then the Inter1, and finally the Inter2 constructor executes */
9
10 int main() {
11     Child obj(10,20);
12     cout<<"Data in Child: " <<obj.getData()<<endl;
13     return 0;
14 }
```

```
Parent Class: 0
Inter1 Class: 0
Inter2 Class: 0
Child Class: 0
Data in Child: 0
```

Example 8

```
1 class Parent {
2     protected:
3         int data;
4     public:
5         Parent(int value) : data(value)
6             {cout<<"Parent Class: "<<data<<endl;} // shared
7 };
8
9 class Inter1 : virtual public Parent {
10     public:
11         Inter1(int value):Parent(value)
12             {cout<<"Inter1 Class:"<<data<<endl;}
13 };
14
15 class Inter2 : public Parent {
16     public:
17         Inter2(int value):Parent(value)
18             {cout<<"Inter2 Class:"<<data<<endl;}
19 };
```

Example 8 (Cont.)

```
1
2 class Child : public Inter1, public Inter2 {
3     public:
4         Child(int a, int b):Inter2(b),Inter1(a),Parent(0)
5         {cout<<" Child Class: "<<data<<endl;} // Ambiguous
6         int getData(){return data;}
7 };
8
9 int main() {
10     Child obj(10,20);
11     cout<<"Data in Child: "<<obj.getData()<<endl;
12     return 0;
13 }
```

Scenario 9.4: Solution to the Diamond Problem

```
1 class Person {
2     string name;
3     int age;
4
5     public:
6         Person(string name, int age) : name(name), age(age) {}
7
8         void setName(string name) { this->name = name; }
9         void setAge(int age) { this->age = age; }
10
11        string getName() const { return name; }
12        int getAge() const { return age; }
13
14        void printInfo() const {
15            cout << "Name: " << name << "\n";
16            cout << "Age: " << age << "\n";
17        }
18 };
```

Scenario 9.4 (Cont.)

```
1 class Student : virtual public Person {
2     int studentId;
3     string grade;
4
5 public:
6     Student(string name, int age, int Id, string grade):
7         Person(name, age), studentId(Id), grade(grade) {}
8
9     void setStudentId(int Id) { studentId = Id; }
10    void setGrade(string grade) { this->grade = grade; }
11
12    int getStudentId() const { return studentId; }
13    string getGrade() const { return grade; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Student ID: " << studentId << "\n";
18        cout << "Grade: " << grade << "\n";
19    }
20};
```

Scenario 9.4 (Cont.)

```
1 class Worker : virtual public Person {
2     string jobTitle;
3     double salary;
4
5 public:
6     Worker(string name, int age, string job, double sal):
7         Person(name, age), jobTitle(job), salary(sal) {}
8
9     void setJobTitle(string job) { jobTitle = job; }
10    void setSalary(double salary) { this->salary = salary; }
11
12    string getJobTitle() const { return jobTitle; }
13    double getSalary() const { return salary; }
14
15    void printInfo() const {
16        Person::printInfo(); // Call base class method
17        cout << "Job Title: " << jobTitle << "\n";
18        cout << "Salary: $" << salary << "\n";
19    }
20};
```

Scenario 9.4 (Cont.)

```
1 class WorkingStudent : public Student, public Worker {
2 public:
3     WorkingStudent(string name, int age, string job, double
        sal, int Id, string grade):Person(name, age),
        Student(name, age, Id, grade), Worker(name, age, job
        , sal) {}
4
5     void printInfo() const {
6         Person::printInfo(); // No ambiguity due to virtual
            inheritance
7         cout << "Student ID: " << getStudentId() << "\n";
8         cout << "Grade: " << getGrade() << "\n";
9         cout << "Job Title: " << getJobTitle() << "\n";
10        cout << "Salary: $" << getSalary() << "\n";
11    }
12 };
```

Scenario 9.4 (Cont.)

```
1 int main() {  
2  
3     WorkingStudent WS("Imran Ali",25,"Programmer",750,8288151,"A");  
4     WS.printInfo();  
5  
6 return 0;  
7 }
```

Name: Imran Ali
Age: 25
Student ID: 8288151
Grade: A
Job Title: Software Engineer
Salary: \$750

Self Test 1

```
1 class X {
2     int x;
3 public:
4     X(int xx) {x = xx; cout<<"X ="<<x<<endl;}
5 };
6 class A : public X {
7 public:
8     A(int aa) : X(aa) { cout << "*A*\n"; }
9 };
10 class B : public X {
11 public:
12     B(int bb) : X(bb) { cout << "*B*\n"; }
13 };
14 class C : public A, public B {
15 public:
16     C(int aa, int bb, int cc):A(aa),B(bb){cout<<"*C*\n";}
17 };
18 int main() {
19     C obj(1, 2, 3);
20     return 0;
21 }
```

```
X = 1
*A*
X = 2
*B*
*C*
```

Self Test 2

```
1 class X {
2     int x;
3 public:
4     X(int xx) {x = xx; cout<<"X ="<<x<<endl;}
5 };
6 class A : public X {
7 public:
8     A(int aa) : X(aa) { cout << "*A*\n"; }
9 };
10 class B : public X {
11 public:
12     B(int bb) : X(bb) { cout << "*B*\n"; }
13 };
14 class C : public A, public B {
15 public:
16     C(int aa, int bb, int cc): X(cc), A(aa),B(bb)
17         {cout<<"*C*\n"; } // Error! X is not direct base of C
18 };
19 int main() {
20     C obj(1, 2, 3);
21     return 0;
22 }
```

Self Test 3

```
1 class X {
2     int x;
3 public:
4     X(int xx) {x = xx; cout<<"X ="<<x<<endl;}
5 };
6 class A : virtual public X {
7 public:
8     A(int aa) : X(aa) { cout << "*A*\n"; }
9 };
10 class B : virtual public X {
11 public:
12     B(int bb) : X(bb) { cout << "*B*\n"; }
13 };
14 class C : public A, public B {
15 public:
16     C(int aa, int bb, int cc): A(aa),B(bb)
17         {cout<<"*C*\n"; } // Error! X must be called from C
18 };
19 int main() {
20     C obj(1, 2, 3);
21     return 0;
22 }
```

Self Test 4

```
1 class X {
2     int x;
3 public:
4     X(int xx) {x = xx; cout<<"X ="<<x<<endl;}
5 };
6 class A : virtual public X {
7 public:
8     A(int aa) : X(aa) { cout << "*A*\n"; }
9 };
10 class B : virtual public X {
11 public:
12     B(int bb) : X(bb) { cout << "*B*\n"; }
13 };
14 class C : public A, public B {
15 public:
16     C(int aa, int bb, int cc): X(cc), A(aa),B(bb)
17         {cout<<"*C*\n"; } // OK!
18 };
19 int main() {
20     C obj(1, 2, 3);
21     return 0;
22 }
```

X = 3
A
B
C

Self Test 5

```
1 class X {
2     public:
3         int x;
4         X(int xx) { x = xx; }
5 };
6 class A : public X {
7     public:
8         A(int aa) : X(aa) {}
9 };
10 class B : public X {
11     public:
12         B(int bb) : X(bb) {}
13 };
14 class C : public A, public B {
15     public:
16         C(int aa, int bb) : A(aa), B(bb) {}
17 };
18 int main() {
19     C obj(1, 2);    //OK! No access to Ambiguous 'x'
20     return 0;
21 }
```

Self Test 6

```
1  class X {
2      public:
3          int x;
4          X(int xx) { x = xx; }
5  };
6  class A : public X {
7      public:
8          A(int aa) : X(aa) {}
9  };
10 class B : public X {
11     public:
12         B(int bb) : X(bb) {}
13 };
14 class C : public A, public B {
15     public:
16         C(int aa, int bb) : A(aa), B(bb) {}
17         void display() {cout<<"x: " <<x<<endl;} // 'x' is Ambiguous
18 };
19 int main() {
20     C obj(1, 2); obj.display(); //Error! Ambiguous access to 'x'
21     return 0;
22 }
```

Conclusions

In this lecture we discussed about

- Multiple Inheritance
- Constructors in Multiple Inheritance
- Virtual Base Classes
- Constructors in Virtual Base Classes

In the next lecture we will discuss more about

- Class Relationships